

Systems Engineering Masterclass

From First Principles to AI Infrastructure

Personal handoff / continuation document

SYSTEMS ENGINEERING MASTERCLASS — CONTINUATION / CONTEXT FILE

Session startup: Read `metadata/AGENT_CONTEXT.md` first. It contains the current operating rules and append-only log of every user suggestion, correction, preference, and workflow update. This file remains the detailed historical and learning-progress handoff.

0. AUTHORITY PIVOT — 2026-09-15 (READ THIS FIRST)

As of 2026-09-15, the user introduced a separate, much more detailed planning artifact: `/home/jd/metadata/curriculum-tracker.docx` — a formal 66-week, 18-stage curriculum tracker for learner "Jeevan" (10-12 hrs/week), with per-week Learning Objectives / Core Topics / Hands-on Work / Weekly Evidence / a single Mastery Checkpoint question / a reflection table, four-week reviews, a capstone portfolio checklist, and a final self-assessment rubric. Its completion standard: every concept must pass **Explain, Draw, Observe (on real Linux), Build/simulate**.

Explicit user decision (2026-09-15): the .docx tracker is now the AUTHORITATIVE plan. This metadata/MASTERCLASS.md file becomes the supporting narrative/session log — teaching style, Why?-notebook, book references, and detailed session-by-session record of what was actually taught/discovered — but the **week numbering, ordering, and mastery checkpoints in the .docx are what we follow**.

Explicit user decision on backfill: start from the beginning. Everything below in §1-§35 (the old "Chapter 1-6" numbering, the practical memory lab) was useful groundwork and real hands-on evidence (see §34), but it was done **out of the tracker's intended order** (it jumped straight from CPU execution to Week-26/27/28-equivalent virtual memory content, skipping the tracker's Weeks 8-25). Per the user's explicit instruction, we are **not** backfilling or crediting that work against tracker weeks — we restart cleanly at **Week 1: Electricity, States, Bits, and Information** and progress through the tracker in order, week by week.

Old §1-§35 content is preserved below for historical/style context only (teaching philosophy, user background, Why?-notebook habits, resource list) — but do NOT treat the old "Chapter N" checkpoints as current progress markers anymore. **Current progress state lives in §36 onward.**

How to run each week (merging tracker rigor with the established teaching style from §2)

For each tracker week:

1. Teach the Core Topics using the mentor/first-principles style (§2-§3, §22-§23) — why before how, diagrams, connect to prior weeks.
2. Explicitly hit every Learning Objective for that week.
3. Do the Hands-on Work for real on this VM (`~/sysprog/` or wherever fits) wherever it's executable — capture real command output/evidence, not just described theory.
4. Walk through the Weekly Evidence checklist.
5. Answer the week's single Mastery Checkpoint question explicitly and explicitly confirm the user can answer it unprompted too.
6. The reflection table (strongest understanding / remaining confusion / connection to work / next revision action) and the Confidence /5 score, Start/End dates, Hours, and Status checkboxes in the .docx are the USER's own honest self-assessment — do not fill these in unprompted; prompt the user to fill them in themselves (or fill them in only if the user explicitly dictates the values).
7. At the end of every 4-week stage-group per the tracker's Four-Week Review sections, do the review explicitly before moving on.

PURPOSE

This is the persistent handoff document for a long-running learning journey:

If this file is provided in a new conversation, treat it as the authoritative context for continuing the course.

The goal is NOT merely interview preparation. The primary goal is to build a deep, connected mental model of computing:

physics → transistors → digital logic → CPU → machine code → assembly → C → compiler → memory → operating system → Linux → kernel → drivers → virtualization → networking → distributed systems → cloud → AI infrastructure

Interview preparation is a secondary benefit.

1. USER'S LEARNING GOAL

The user explicitly wants to understand:

"everything from scratch and need to understand how everything i.e. an OS works and how exactly computation occurs."

The user feels they have lost grip on fundamentals despite currently working in Linux/kernel-related work.

Do NOT interpret this as lack of practical ability. The user already has real kernel/system experience. The problem is that foundational concepts feel disconnected.

The desired outcome is a coherent mental model, not memorized definitions.

The user strongly responded positively to this approach and said:

"This is the exact thing I was looking for !!!!!!!!!!!!!!!1"

2. HOW TO TEACH THIS USER

Teach like an experienced systems mentor sitting at a whiteboard, NOT like a generic interview-prep chatbot.

The user wants:

- First-principles explanations.
- "WHY?" before "HOW?"
- Connections between abstraction layers.
- Hardware → software relationships.
- Historical motivation where useful.
- Concrete examples.
- Execution traces.
- Diagrams.
- Hands-on experiments.
- C and assembly when appropriate.
- Linux/kernel source eventually.
- QEMU/KVM experiments.
- Driver development later.
- Interview questions only as secondary reinforcement.

Response style

The current style is:

1. Start from a simple question.
2. Explain the underlying concept.
3. Ask "why?"
4. Build the next abstraction.
5. Connect it to previous concepts.
6. Show a conceptual ASCII diagram.
7. Give a few "Why?" questions.
8. End with the next conceptual bridge.

Continue this style.

Do NOT suddenly switch to:

- generic textbook summaries,
- shallow interview answers,
- huge walls of terminology,
- assuming advanced knowledge without deriving it.

The user likes detailed, enthusiastic, conceptual teaching.

3. CORE TEACHING RULE

The user asks questions such as:

"But where does that actually exist?"

These questions should be encouraged.

When explaining an abstraction, distinguish carefully between:

- specification
- physical implementation
- representation/encoding
- runtime state
- software implementation

Example already discussed:

"Where is the instruction set stored?"

Correct distinction:

1. **ISA specification** — architecture documentation / contract.
2. **Instruction encoding** — machine-code bytes stored in executable/memory.
3. **Instruction decoder and execution behavior** — implemented in CPU hardware/control machinery.
4. Modern CPUs may decode complex instructions into internal μ ops.

Do not say "the instruction set is stored in RAM."

4. MASTERCLASS CURRICULUM

LEVEL 0 — NATURE OF COMPUTATION

Goal: What is computation?

Topics:

- History of computation
- Information
- State
- Binary
- Bits and bytes
- Number systems
- Boolean reasoning
- Algorithms vs computation
- Why digital computation works

Projects:

- Binary calculator
- Decimal ↔ binary converter
- Hex tools

LEVEL 1 — DIGITAL ELECTRONICS

Goal: How does electricity become logic?

Topics:

- Electricity
- Voltage
- Current
- Resistance
- Semiconductor basics
- Transistors
- MOSFET
- CMOS
- Logic gates
- NOT, AND, OR, XOR, NAND, NOR
- Truth tables
- Latches
- Flip-flops
- Registers
- Multiplexers
- Decoders
- Encoders
- Adders
- ALU

Projects:

- Logic gate simulator
- Half adder
- Full adder
- 8-bit ALU

LEVEL 2 — COMPUTER ARCHITECTURE

Goal: How does a CPU execute instructions?

Topics:

- CPU organization
- Registers
- Program Counter / Instruction Pointer
- ALU
- Control unit
- Instruction cycle
- Fetch
- Decode
- Execute
- Memory access
- Writeback
- ISA
- Machine code
- Instruction encoding
- x86-64
- ARM overview
- Memory hierarchy
- SRAM
- DRAM
- Cache
- Cache lines
- Locality
- Pipeline
- Branch prediction
- Superscalar execution
- Out-of-order execution
- Register renaming
- Speculation
- SIMD
- Performance basics
- Architectural vs microarchitectural state

Projects:

- Simple CPU simulator
- Cache simulator

LEVEL 3 — ASSEMBLY LANGUAGE

Goal: What does the CPU actually execute?

Topics:

- Machine code
- Assembly
- x86-64 registers
- General-purpose registers
- RIP
- RSP
- RBP
- RFLAGS
- Calling convention

- Stack
- Stack frames
- Function calls
- call/ret
- push/pop
- Loads/stores
- Arithmetic
- Branches
- Addressing modes
- Exceptions
- Interrupts

Projects:

- C ↔ Assembly interoperation
- Small assembly programs
- GDB instruction-level tracing

LEVEL 4 — COMPILERS AND TOOLCHAIN

Goal: How does C become machine code?

Topics:

- Preprocessor
- Compiler
- Lexer
- Parser
- AST
- Optimization
- Code generation
- Assembly output
- Assembler
- Object files
- ELF
- Symbols
- Relocations
- Linker
- Dynamic linker
- Loader
- Shared libraries
- ABI

Projects:

- ELF parser
- Toy compiler
- Compile C with -S and -c
- Inspect with objdump/readelf/nm

LEVEL 5 — C PROGRAMMING MASTERY

Goal: Understand C as a systems language and understand what the compiler/hardware actually do.

Topics:

- Types
- Integer representation
- Signed/unsigned

- Two's complement
- Pointers
- Pointer arithmetic
- Arrays vs pointers
- Strings
- Structures
- Unions
- Padding
- Alignment
- const
- volatile
- restrict
- Storage classes
- Scope
- Lifetime
- Stack
- Heap
- malloc/calloc/realloc/free
- Function pointers
- Callbacks
- Macros
- Preprocessor
- Bit operations
- Undefined behavior
- Integer overflow
- Aliasing
- Memory model
- Concurrency
- Threads
- POSIX APIs

Projects:

- Custom malloc
- Dynamic array
- Hash table
- Thread pool
- Memory debugging tool

LEVEL 6 — OPERATING SYSTEMS

Goal: Why do operating systems exist and how do they work?

Topics:

- What an OS provides
- Kernel vs user space
- Privilege levels
- Processes
- Process address space
- Threads
- Context switching
- Scheduling
- Synchronization
- Mutexes
- Spinlocks
- Semaphores

- Condition variables
- Deadlocks
- IPC
- Pipes
- Shared memory
- Signals
- System calls
- Virtual memory
- Page tables
- Paging
- Page faults
- TLB
- Memory allocation
- Filesystems
- File descriptors
- I/O
- Security
- Protection

Projects:

- Mini shell
- Mini scheduler
- Mini filesystem
- Mini allocator
- Mini virtual memory simulator

LEVEL 7 — LINUX SYSTEM PROGRAMMING

Goal: How do applications interact with Linux?

Topics:

- System calls
- fork
- exec
- wait
- mmap
- brk
- open/read/write/close
- ioctl
- poll/select/epoll
- Signals
- Pipes/FIFOs
- POSIX shared memory
- POSIX semaphores
- pthreads
- Sockets
- TCP/UDP
- File descriptors

Projects:

- Shell
- Multithreaded server
- epoll server
- Chat server
- IPC applications

- Thread pool

LEVEL 8 — LINUX KERNEL

Goal: How is Linux actually implemented?

Topics:

- Linux boot
- Kernel initialization
- Kernel/user boundary
- System call entry
- Process representation
- task_struct
- Scheduler
- Context switching
- Kernel threads
- Workqueues
- Interrupts
- Softirqs
- Locking
- Spinlocks
- Mutexes
- RCU
- Memory management
- Buddy allocator
- SLAB/SLUB
- Page cache
- VFS
- Filesystems
- Networking stack
- Device model
- Sysfs
- Procfs
- Debugfs
- Kernel modules
- Tracing
- ftrace
- perf
- printk
- Dynamic debug
- NUMA
- CPU topology

Projects:

- Kernel module
- Character driver
- procfs entry
- sysfs interface
- scheduler experiment
- memory-management experiment

5. DRIVER DEVELOPMENT TRACK

Driver development is an explicit user goal and should be integrated into the masterclass.

Do NOT leave drivers until the very end.

Progression:

Driver 1 — Kernel modules

- module_init
- module_exit
- module parameters
- module loading/unloading
- symbols
- logging

Driver 2 — Character devices

- device numbers
- cdev
- file_operations
- open/read/write/ioctl/poll/mmap

Driver 3 — Kernel/user boundary

- copy_to_user
- copy_from_user
- permissions
- blocking/nonblocking I/O

Driver 4 — Interrupts

- IRQ
- threaded interrupts
- interrupt context
- deferred work
- synchronization

Driver 5 — MMIO

- memory-mapped I/O
- register access
- barriers
- ordering

Driver 6

GPIO

Driver 7

I2C

Driver 8

SPI

Driver 9

DMA

Driver 10 — PCIe

- enumeration
- BARs
- MMIO
- MSI/MSI-X
- DMA
- IOMMU
- driver binding

The Raspberry Pi 4 is a future physical hardware lab for GPIO/I2C/SPI/interrupt experiments, but it is NOT required now.

6. HARDWARE TRACK

Topics:

- PCIe
- DMA
- IOMMU
- Interrupt controllers
- Storage
- NVMe
- USB
- GPU
- Memory controller
- NUMA
- Cache coherency
- Memory ordering

Connect this directly to driver development and Linux kernel internals.

7. VIRTUALIZATION

Especially relevant to the user's current experience.

Topics:

- Virtual machines
- Hypervisors
- QEMU
- KVM
- vCPU
- VM exits
- VM entries
- Virtual memory
- Shadow paging concepts
- EPT/NPT
- Intel VMX
- AMD SVM
- Nested virtualization

- IOMMU
- Device virtualization
- virtio
- QEMU device model
- Confidential computing
- AMD SEV
- SEV-ES
- SEV-SNP
- Attestation
- RMP

The user already has practical KVM/QEMU/SEV-SNP/kernel-backport experience. Eventually connect the first-principles material to that experience.

8. NETWORKING

Topics:

- Ethernet
 - MAC
 - ARP
 - IP
 - Routing
 - TCP
 - UDP
 - TCP state machine
 - Sockets
 - DNS
 - HTTP
 - TLS
 - NICs
 - Interrupts
 - NAPI
 - DMA
 - Zero-copy
 - RDMA
 - DPDK
 - eBPF
-

9. DISTRIBUTED SYSTEMS

Topics:

- RPC
- gRPC
- CAP
- Consistency
- Replication
- Leader election
- Consensus
- Raft
- Sharding
- Partitioning

- Distributed locking
 - Service discovery
 - Load balancing
 - Kafka
 - etcd
 - Observability
 - Failure handling
-

10. CONTAINERS AND CLOUD

Topics:

- Linux namespaces
 - cgroups
 - capabilities
 - seccomp
 - container runtime
 - Docker
 - Kubernetes
 - Pods
 - Services
 - Deployments
 - Scheduling
 - Control plane
 - Networking
 - Storage
 - Observability
-

11. AI INFRASTRUCTURE

Topics:

- GPU architecture
- CPU/GPU relationship
- CUDA concepts
- GPU memory
- PCIe
- NVLink
- NCCL
- RDMA
- GPU scheduling
- Slurm
- Kubernetes GPU scheduling
- GPU control plane
- GPU data plane
- training infrastructure
- inference infrastructure
- large-scale cluster reliability

This connects to the Oracle OCI AI Infrastructure role the user is considering.

12. SYSTEM DESIGN

Ongoing:

- Distributed cache
 - Scheduler
 - Filesystem
 - Logging system
 - Monitoring system
 - Load balancer
 - Cloud storage
 - GPU scheduler
 - AI cluster control plane
-

13. CURRENT PROGRESS

Completed

Chapter 1 — What is Computation?

Covered:

- computation as transformation of information according to rules
- information
- state
- physical states
- why computers exploit physical state
- binary motivation

Chapter 2 — Why Binary?

Covered:

- bits
- binary states
- combinations of bits
- 2^n possible combinations
- storage vs processing
- need for logic

Chapter 3 — Logic Gates

Covered:

- NOT
- AND
- OR
- XOR
- NAND
- truth tables
- logic gates as computation
- transistors → gates
- half adder
- sum = XOR

- carry = AND
- path toward ALU

Chapter 4 — How Does a Computer Remember?

Covered:

- combinational vs sequential logic
- state
- feedback
- latches
- flip-flops
- clock
- registers
- why CPU needs registers
- control
- instructions
- Program Counter / instruction pointer

Chapter 5 — How Does the CPU Actually Execute an Instruction?

Covered:

- instruction
- opcode concept
- operands
- instruction stored in memory
- RIP / Program Counter
- fetch
- decode
- register read
- execute
- writeback
- update instruction pointer
- cache as part of instruction fetch
- pipeline concept
- architectural vs microarchitectural state
- C → compiler → machine code → CPU
- machine instructions vs internal μ ops

Follow-up: Invalid Instruction

Discussed:

- CPU decoder examines instruction bytes
- invalid encoding on x86 can produce #UD
- CPU enters exception handling
- Linux handles it
- normal user-space process can receive SIGILL
- invalid instruction is different from a valid but prohibited instruction and from a memory fault

14. IMMEDIATE NEXT LESSON (superseded — see §32/§34 for latest checkpoint)

Chapter 6 — What Does It Mean to Access Memory?

Start from:

```
int *p = NULL;
*p = 10;
```

The instruction is valid, so why does the program fault?

Teach from first principles:

1. What is a memory address?
2. What does 0x0 mean?
3. CPU-generated address
4. Virtual address
5. Physical address
6. Why virtual memory exists
7. MMU
8. Page tables
9. TLB
10. Page-table lookup
11. Present/not-present
12. Permission bits
13. Page fault exception
14. Linux page-fault handling
15. Why NULL is normally unmapped
16. User vs kernel address spaces
17. Why a page fault is not always a bug
18. Demand paging
19. Copy-on-write
20. Stack growth
21. Memory-mapped files
22. Fatal invalid access
23. Connect to malloc(), mmap(), and Linux memory management

Do not jump into complicated page-table structures before the virtual-vs-physical distinction is intuitive.

15. IMPORTANT LAB DECISION

Current decision:

Use QEMU/KVM for now.

Do not require Raspberry Pi yet.

The user considered buying a Raspberry Pi 4 and plans to learn driver development. We concluded that a Pi 4 would be useful later for:

- GPIO

- I2C
- SPI
- interrupts
- physical hardware interaction

For now:

```
Host Linux
  ↓
KVM
  ↓
QEMU
  ↓
Guest Linux
  ↓
Our experiments
```

Do not abruptly jump into setup unless the lesson reaches a point where hands-on work is useful.

16. FUTURE QEMU/KVM LAB

Level 1 — Toolchain

- gcc
- clang
- gdb
- objdump
- readelf
- nm
- strace
- perf
- ftrace

Level 2 — CPU

- assembly
- registers
- instructions
- calling convention
- stack
- ELF

Level 3 — OS

- processes
- threads
- virtual memory
- system calls
- IPC

Level 4 — Kernel

- build kernel
- boot own kernel
- modules
- printk
- procfs
- sysfs

- tracing

Level 5 — Drivers

- character driver
- virtual hardware
- interrupts
- MMIO
- QEMU devices

Level 6 — Virtualization

- QEMU
- KVM
- vCPU
- VM exit
- guest/host interaction
- memory virtualization
- IOMMU
- SEV/SNP

17. USER BACKGROUND — CALIBRATION ONLY

The user has approximately 3 years total professional experience:

- ~8 months prior IBM experience
- ~2.6 years current role at the time of career planning

Experience includes:

- Linux kernel
- kernel backporting
- KVM/QEMU
- AMD EPYC platform enablement
- AMD SEV/SEV-SNP
- confidential computing
- kernel memory management
- MCE/EDAC-related work
- virtualization
- enterprise Linux
- kernel validation
- QEMU/EDK2
- Git
- LKP testing
- Linux system programming

The user has worked on upstream/backport patches and understands advanced kernel concepts in practice.

However, the user explicitly feels they have lost grip on basics.

Therefore:

- Do NOT assume foundational understanding just because they work in kernel.
- Do NOT talk down to them.
- Explain from first principles.
- Connect to their existing experience when useful.

18. CAREER OBJECTIVE

The user is preparing for a job switch.

Target directions:

- Linux Kernel Engineer
- Systems Software Engineer
- Platform Software Engineer
- Infrastructure Software Engineer
- Cloud Infrastructure Engineer
- AI Infrastructure Engineer

The user considered an Oracle OCI AI Infrastructure role involving:

- GPU control plane
- GPU data plane
- distributed systems
- large-scale AI infrastructure
- C/C++/Go/Rust/etc.
- Kubernetes/Slurm
- cloud infrastructure

We assessed it conceptually as a strong-but-not-perfect match, with gaps in:

- 5+ years requirement
- distributed systems
- cloud infrastructure
- Kubernetes
- large-scale services
- technical leadership

Career preparation is secondary to the deeper systems-engineering masterclass.

19. CORE RESOURCES

Recommended books:

1. **Computer Systems: A Programmer's Perspective (CS:APP)** — primary backbone.
2. **Operating Systems: Three Easy Pieces (OSTEP)** — OS concepts.
3. **The Linux Programming Interface (TLPI)** — Linux system programming.
4. **Linux Kernel Development** — kernel architecture.
5. **Understanding the Linux Kernel** — deeper reference.
6. **Linux Device Drivers** — selected/legacy reference; prefer modern kernel documentation/source where APIs differ.
7. **Intel 64 and IA-32 Architectures Software Developer's Manual** — selected sections.

Do NOT tell the user to read all books cover-to-cover.

Use them selectively when relevant.

20. STUDY METHOD

The user asked whether C and OS should be mixed every day.

We agreed that mixing related subjects is better than isolated blocks because:

- pointers connect to addresses
- C memory connects to virtual memory
- threads connect to scheduling
- synchronization connects to kernel locks
- malloc connects to memory management
- system calls connect user space to kernel

Eventually use a daily mix of:

- C / architecture
- OS / Linux
- coding/experiment
- reading
- debugging/kernel exploration
- "Why?" notebook

During the earliest hardware-first chapters, follow conceptual dependencies rather than forcing C/OS into every lesson.

21. WHY NOTEBOOK

Maintain questions the user naturally asks.

Examples:

- Why is binary used?
- Why does adding one bit double possible values?
- Why can't switches alone compute?
- Why do logic gates work?
- Why does XOR produce the sum bit?
- Why does AND produce carry?
- Why can't combinational logic remember?
- Why does CPU need registers?
- Why does CPU need a clock?
- Why does CPU need Program Counter?
- Why doesn't 3 GHz mean exactly 3 billion instructions/sec?
- Where is the ISA stored?
- How does CPU know an instruction encoding means ADD?
- What happens when instruction bytes are invalid?
- Why does a valid instruction sometimes cause an exception?
- What exactly happens when `*p = 10` and `p == NULL`?

These questions are not distractions. They are the learning path.

22. RESPONSE STYLE FOR "NEXT"

When the user says:

"next"

Continue to the next logical chapter without unnecessary confirmation.

Use:

Chapter N — Title

Then:

- motivating question
- first-principles explanation
- progressive derivation
- conceptual diagrams
- connection to previous lessons
- mental model
- Why? questions
- next chapter preview

Avoid shallow definitions.

23. DO NOT BREAK THE LEARNING CONTRACT

The course is based on:

Never memorize something until you understand why it exists.

Examples:

- Do not just say "stack grows downward"; explain that this is convention/ABI/implementation and not a universal law.
 - Do not just say "page fault = error"; explain that page faults are a normal CPU exception mechanism and can support demand paging/COW.
 - Do not just say "instruction set is stored in CPU"; distinguish ISA specification, machine-code encoding, and hardware implementation.
 - Do not just say "malloc allocates memory"; trace user space → libc → system calls → kernel → virtual memory → physical memory as appropriate.
-

24. ULTIMATE MENTAL MODEL

The eventual end-to-end model should look approximately like:

```
Physical world
↓
Electricity
↓
Semiconductors
↓
Transistors
↓
Logic gates
↓
Sequential logic
↓
Registers
↓
ALU
↓
CPU
↓
ISA / machine instructions
↓
Assembly
↓
Compiler
↓
C
↓
Executable / ELF
↓
Loader
↓
Process
↓
Virtual memory
↓
System calls
↓
Linux kernel
↓
Kernel subsystems
↓
Drivers
↓
Hardware
↓
Virtualization / containers
↓
Networking
↓
Distributed systems
↓
Cloud
↓
AI infrastructure
↓
Thousands of CPUs/GPUs
```

The user wants this as a connected system, not as independent subjects.

25. HANDOFF INSTRUCTION FOR A NEW CHAT

If this file is uploaded in a new conversation, treat it as the continuation of the same masterclass.

Start by briefly acknowledging:

"I have the masterclass context. We are currently at Chapter 6: memory addressing and the NULL pointer/page-fault bridge."

Then continue from the exact next lesson unless the user asks to change direction.

Do NOT:

- restart the curriculum
- re-explain the entire history
- ask the user to repeat their goals
- change the teaching style

Preserve:

- first-principles approach
- whiteboard/mentor style
- "WHY?" emphasis
- layered execution tracing
- detailed conceptual diagrams
- hands-on/QEMU/KVM orientation
- connection to Linux/kernel/driver work

The new conversation should feel like the same teacher continuing the same course.

26. CHECKPOINT

Masterclass handoff version: **v1.2**

Current level: **Level 2 / Chapter 6 — Memory addressing and virtual memory bridge**

Completed:

- Chapter 1 — What is Computation?
- Chapter 2 — Why Binary?
- Chapter 3 — Logic Gates
- Chapter 4 — How Does a Computer Remember?
- Chapter 5 — How Does the CPU Actually Execute an Instruction?
- Follow-up — Invalid instruction / #UD / SIGILL

Next:

Practical Chapter 6 Memory Lab — observe NULL faults, mappings, `malloc()` / `mmap()` , page touching, and page-fault behavior.

Current lab:

QEMU/KVM first; Raspberry Pi later.

Driver development:

Explicit future track.

Core objective:

27. LEARN-BY-DOING STRATEGY — UPDATED 2026-09-03

The course strategy has been deliberately changed from reading-heavy progression to an experiment-driven model. The user has a full-time office schedule (9:00 AM–6:00 PM) with roughly 1–2 hours available in the morning and 1–2 hours in the afternoon.

Preferred daily structure:

- Morning: concepts, reading, first-principles derivation, diagrams, and mental-model building.
- Afternoon: hands-on experiments, coding, tracing, debugging, QEMU/KVM work, and kernel exploration.
- Evening: optional 15–20 minute consolidation / Why? notebook update.

Target balance:

30–40% learning, 60–70% doing.

The teaching loop should be:

Question → learn just enough → implement → observe → break → debug → explain → connect to Linux.

Do not make the user read textbooks cover-to-cover. Books are references used to support the current experiment.

28. CURRENT UNDERSTANDING / PROGRESS

Completed chapters remain:

- Chapter 1 — What is Computation?
- Chapter 2 — Why Binary?
- Chapter 3 — Logic Gates
- Chapter 4 — How Does a Computer Remember?
- Chapter 5 — How Does the CPU Actually Execute an Instruction?
- Follow-up — Invalid instruction / #UD / SIGILL

The user is now transitioning into the memory/addressing bridge and wants the concepts grounded in actual experiments rather than passive study.

29. CURRENT LAB STATUS

Primary laboratory environment:

QEMU/KVM first; Raspberry Pi later.

The hands-on track should increasingly use small C programs, GCC/Clang, GDB, objdump, readelf, strace, perf/fttrace, QEMU/KVM, and eventually Linux kernel source and modules.

The lab progression should move from user-space observation into kernel tracing and then driver development:

- address/memory experiments
- processes and system calls
- virtual memory/page faults
- kernel build/boot/module experiments
- character driver
- virtual hardware

- interrupts/MMIO/DMA/PCIe
- QEMU device experiments

Driver development remains an explicit future track, not yet the immediate lesson.

30. OPEN WHY? QUESTIONS / ACTIVE INVESTIGATION

Preserve the existing Why? notebook and add/track questions that arise during experiments. Current memory-focused questions include:

- What exactly happens when `int *p = NULL; *p = 10; ?`
- What does address `0x0` mean at the CPU/MMU level?
- Where does a virtual address actually exist?
- Who translates virtual → physical, and when?
- What does the page table physically contain?
- What exactly does the TLB cache?
- Why can a page fault be normal rather than an error?
- What changes when a program touches a newly allocated page?
- How do `malloc()` and `mmap()` connect user space to the kernel's virtual-memory machinery?

These questions are learning checkpoints, not a backlog to memorize.

31. EXACT NEXT LESSON CHECKPOINT (superseded — see §34)

Resume here:

Chapter 6 — What does it actually mean for the CPU to access memory address `0x0`, and why does `int *p = NULL; *p = 10;` fault?

The next session should NOT jump directly to a broad RAM/cache lecture. First complete the memory-address/page-fault bridge with a concrete experiment.

Next morning

Derive, from first principles:

1. C pointer vs. numeric address
2. virtual address vs. physical address
3. MMU role
4. page tables
5. TLB
6. page fault as a CPU exception mechanism
7. why NULL access normally faults in user space

Next afternoon

Hands-on experiment:

1. Write a tiny C program involving `NULL`, `malloc()`, and `mmap()`.
2. Compile with debug symbols and suitable optimization settings.
3. Inspect the process mappings through `/proc/<pid>/maps`.
4. Use GDB to inspect addresses and the faulting instruction.
5. Use tracing/tools as appropriate to observe page-fault/system-call behavior.
6. Compare what was expected with what actually happened.
7. Explain the full path without notes.

Only after this checkpoint is understood should the course bridge into **RAM organization and CPU caches → SRAM → CPU cache → DRAM → memory controller → RAM.**

32. AUTOMATION CHECKPOINT — 2026-09-05

No new completed masterclass lesson or lab execution has been recorded since the previous checkpoint. The handoff remains intentionally pinned to the practical Chapter 6 memory lab rather than advancing the curriculum prematurely.

Current state:

- Conceptual Chapter 6 memory/addressing material is complete enough to experiment.
- Practical memory lab is still pending.
- No cache/RAM lesson should be treated as completed yet.
- QEMU/KVM remains the primary lab environment; Raspberry Pi remains deferred.
- The next meaningful update should record the actual commands run, observations, page-fault behavior, and any misconceptions corrected during the memory lab.

Exact resume point:

Run the Chapter 6 practical memory lab first; then bridge the observed behavior into RAM organization, cache lines, locality, and DRAM.

33. HANDOFF RULE

At every meaningful checkpoint, update this file with:

- completed chapters
- current mental model / understanding
- open Why? questions
- lab state and commands/experiments completed
- discoveries or misconceptions corrected
- exact next lesson checkpoint
- next hands-on experiment

The file must remain sufficient for a new conversation to continue the masterclass without restarting or asking the user to reconstruct prior context.

34. LAB EXECUTION LOG — 2026-09-15 — Chapter 6 practical memory lab COMPLETE

Environment: personal practice VM (Fedora Linux 44, on a company network — internet egress to some domains e.g. chatgpt.com is filtered by upstream DPI, irrelevant to this lab). Full C/C++ systems toolchain installed (gcc/clang/gdb/valgrind/strace/perf/bpfttrace/etc.), practice workspace at `~/sysprog/`.

Lab code lives in `~/sysprog/07_memory_lab/`:

- `null_deref.c` — `int *p = NULL; *p = 10;`
- `demand_paging.c` — 10 MB anonymous `mmap()`, touches one byte per page, prints `VmRSS` from `/proc/self/status` before/during/after
- `maps_probe.c` — dumps `/proc/self/maps` before/after a small `malloc(16)` and a large `malloc(1<<20)`
- `Makefile` — builds all three with `-g -O0`

Experiment 1 — NULL pointer write, real evidence captured

Ran directly: shell reported `Segmentation fault (core dumped)`, exit code **139** (128+11, confirming signal 11 = SIGSEGV).

Kernel's own report via `dmesg`:

```
null_deref[250744]: segfault at 0 ip 00000000004004d9 sp 00007ffcc592f060 error 6 in
null_deref[4d9,400000+1000]
Code: ... 48 8b 45 f8 <c7> 00 0a 00 00 00 ...
```

Decoded:

- `segfault at 0` = the faulting **virtual address** was literally `0x0`.
- `error 6` = binary `110` = `bit2(U/S)=1` **user mode**, `bit1(W/R)=1` **write**, `bit0(P)=0` **page not present**. I.e.: "a user-mode write to a non-present page."
- `<c7> 00 0a 00 00 00` = the faulting machine-code bytes = `movl $0xa, (%rax)` — opcode `c7 /0` with immediate `0000000a` = decimal **10**. This is `*p = 10` at the machine-code level, byte for byte.

Cross-checked with `gdb` (breakpoint at the line, `next`, then `info registers`, `disassemble`):

- `rax = 0x0` at the faulting instruction (confirms `p == NULL` loaded into `rax`).
- `rip = 0x4004d9`, disassembly shows `=> movl $0xa, (%rax)` at that exact address — matches the `dmesg` `Code: dump` exactly.
- GDB independently reports "Program received signal SIGSEGV."

Full causal chain established end-to-end with real evidence (not just theory):

```
C source *p = 10 (p==NULL)
-> compiles to movl $0xa, (%rax)    [rax holds 0]
-> CPU executes: generates virtual address 0x0 for the write
-> MMU/page-table walk: no PTE covers address 0 (confirmed no VMA
  starts below the ELF load address 0x400000 in /proc/pid/maps)
-> CPU raises page-fault exception (#PF, vector 14), pushes error
  code 6 (user, write, not-present) and faulting address (CR2) onto
  the trap frame
-> CPU traps into the kernel via the fault gate
-> kernel page-fault handler looks up the VMA for address 0,
  finds none applicable -> decides this cannot be resolved
-> kernel sends SIGSEGV to the process
-> default disposition: terminate + core dump
-> shell observes exit status 139 = 128 + SIGSEGV(11)
```

Experiment 2 — demand paging, RSS growth observed live

Output (real run):

```
RSS before mmap()           = 1776 kB
mmap()'d 10485760 bytes (virtual) at 0x7f2b0ec00000
RSS right after mmap()       = 1840 kB <-- barely changed!
  touched   512 pages so far -> RSS = 3892 kB
  touched  1024 pages so far -> RSS = 5940 kB
  touched  1536 pages so far -> RSS = 7988 kB
  touched  2048 pages so far -> RSS = 10036 kB
RSS after touching ALL pages = 12080 kB
```

$512 \text{ pages} \times 4096 \text{ bytes} = 2,097,152 \text{ bytes} \approx 2048 \text{ kB}$, matching each RSS jump exactly. This is direct proof that:

- `mmap()` only reserves **virtual address space** and creates a VMA; it does **not** immediately commit physical RAM.
- Each first-touch of a page triggers a **minor page fault** (valid VMA, not-present page) that the kernel resolves silently by allocating a physical frame, zeroing it, and installing a PTE — completely invisible to the program (no signal, no crash), then the faulting instruction is **restarted** and succeeds.
- This is the same underlying CPU mechanism (`#PF`) as Experiment 1's crash. The only difference is what the kernel's fault handler decides: VMA found + permissions OK -> fix silently and resume; VMA missing/permissions violated -> deliver SIGSEGV. **A page fault is not inherently an error — it's a general-purpose "trap into the kernel to fix up memory" mechanism.** This directly answers the pinned Why? question: "Why can a page fault be normal rather than an error?"

Experiment 3 — /proc/pid/maps and malloc's brk vs mmap decision

- `malloc(16)` (tiny): `/proc/pid/maps` **unchanged** — served out of already-existing heap arena space, no new syscall visible.
- `malloc(1<<20)` (1 MB, exceeds glibc's default `mmap_threshold` ~128 KB): an existing anonymous `rw-p` VMA adjacent to `libc.so.6` grew by exactly ~1 MB (from a 3-page/12 KB mapping to covering the requested size). The `[heap]` segment itself (`brk` -managed) stayed byte-for-byte identical in both snapshots.
- Confirms: glibc's `malloc()` is a user-space allocator that itself decides, per-request, whether to serve memory from its `brk` -extended heap (small/medium allocations) or hand off to a dedicated `mmap()` (large allocations) — the kernel has no concept of "malloc" at all; it only sees `brk / sbrk` and `mmap` syscalls.
- Also observed: `/proc/pid/maps` **coalesces adjacent VMAs** that share identical permissions/backing into a single line — worth remembering when reading real maps output.

Misconceptions corrected / sharpened this session

- None discovered wrong — but usefully **unified**: "invalid instruction" (#UD, Follow-up after Ch.5) and "invalid memory access" (#PF, this chapter) are now clearly distinguished as two different CPU exception types with two different downstream signals (SIGILL vs SIGSEGV), and page faults were further split into **benign/expected** (demand paging, COW, stack growth) vs **fatal** (no VMA / permission violation) outcomes of the exact same hardware mechanism.

Updated Why? notebook — answered this session

- "What exactly happens when `int *p = NULL; *p = 10; ?`" -> answered end-to-end above, with real `dmesg+gdb` evidence.
- "What does address `0x0` mean at the CPU/MMU level?" -> just another virtual address; the CPU doesn't treat it specially, but the OS/loader convention deliberately never maps a VMA at or near address 0 (a deliberate unmapped "guard" region) precisely so `NULL` derefs reliably fault instead of silently corrupting something.
- "Why can a page fault be normal rather than an error?" -> answered via Experiment 2: same #PF mechanism, outcome depends on kernel fault-handler decision (VMA present+permitted -> silent fix-up; else -> SIGSEGV).
- "What changes when a program touches a newly allocated page?" -> answered via Experiment 2 (RSS growth in exact page-size increments).
- "How do `malloc()` and `mmap()` connect user space to the kernel's virtual-memory machinery?" -> answered via Experiment 3 (`brk`-extended heap vs dedicated `mmap`, threshold-based).

Still open / deferred to later chapters

- What exactly does a page-table entry (PTE) contain bit-by-bit, and how many levels does x86-64 page-table walk have? (deferred to a dedicated page-table-structure chapter, deliberately not front-loaded per original curriculum instruction.)
- What does the TLB cache precisely, and what happens on a TLB miss vs a full page-table walk? (same, deferred.)
- Copy-on-write and stack growth as specific categories of "benign page fault" — same mechanism as Experiment 2, not yet demonstrated with a dedicated experiment.

35. CHECKPOINT (supersedes §26/§31/§32)

Masterclass handoff version: **v1.3**

Current level: **Level 2, end of Chapter 6** — the memory-addressing / page-fault bridge is now conceptually AND experimentally complete (real `dmesg` + `gdb` evidence captured, see §34).

Completed:

- Chapters 1-5 (unchanged, see §13)
- Follow-up — Invalid instruction / #UD / SIGILL (unchanged, see §13)
- **Chapter 6 — Memory addressing, virtual-vs-physical, page faults, demand paging (conceptual + practical lab, §34)**

Next checkpoint (per original curriculum plan in §32, now due):

Chapter 7 — RAM organization and CPU caches: SRAM vs DRAM, the memory controller, cache lines, spatial/temporal locality, and why page-table walks themselves need to be cached (bridging naturally into the still-open PTE-structure and TLB questions from §34).

Suggested next hands-on experiment (afternoon-style, to pair with Chapter 7 once taught):

Write a small C benchmark that strides through a large array with varying stride sizes (sequential vs cache-line-sized vs page-sized-and-larger strides) and times it, to make cache-line/locality effects directly observable (`perf stat -e cache-misses,cache-references` from the toolchain already installed on this VM).

Lab environment: this personal practice VM (`~/sysprog/`), user-space experiments for now; QEMU/KVM and kernel-level work remain queued for Level 8 (§16) as originally planned. Raspberry Pi remains deferred.

Driver development: unchanged, explicit future track (§5).

Core objective: unchanged (§18/§24) — deep first-principles understanding, career prep secondary.

36. CURRENT AUTHORITATIVE PLAN — `.docx` TRACKER (supersedes all prior checkpoints)

Source of truth for ordering/scope: `/home/jd/metadata/curriculum-tracker.docx` (18 stages, 66 weeks — full stage/week map extracted 2026-09-15, reproduced here for offline reference):

Stage 1	(Wk 1-4)	Foundations of Computation
Stage 2	(Wk 5-7)	CPU Architecture and Instruction Execution
Stage 3	(Wk 8-10)	Assembly, ABI, and Program Representation
Stage 4	(Wk 11-13)	Memory Hierarchy and Performance
Stage 5	(Wk 14-18)	C from the Machine's Perspective
Stage 6	(Wk 19-21)	Compilation, Linking, ELF, and Program Startup
Stage 7	(Wk 22-25)	Operating-System Foundations
Stage 8	(Wk 26-29)	Virtual Memory
Stage 9	(Wk 30-33)	Concurrency and Synchronization
Stage 10	(Wk 34-37)	Linux System Programming
Stage 11	(Wk 38-41)	Linux Kernel Internals
Stage 12	(Wk 42-44)	Linux Memory Management
Stage 13	(Wk 45-47)	Devices, Drivers, and I/O
Stage 14	(Wk 48-50)	Filesystems and Storage
Stage 15	(Wk 51-54)	Networking
Stage 16	(Wk 55-57)	Virtualization
Stage 17	(Wk 58-60)	Containers and Kubernetes
Stage 18	(Wk 61-66)	Distributed Systems and Cloud Infrastructure (Wk 66 = GPU/AI infra capstone)

Each week's exact Learning Objectives / Core Topics / Hands-on Work text lives only in the `.docx` — re-read it with the extraction method in §37 if it's ever needed verbatim again (don't hand-copy it into this file; keep this file as the session log, not a duplicate of the tracker).

37. HOW TO RE-EXTRACT THE .DOCX (tooling note)

Direct file-read of `.docx` is not supported by the editor's read tool. Working method (used 2026-09-15):

```

pip3 install --user python-docx # PyPI reachable even though chatgpt.com is not (see networking chat)
python3 - <<'EOF'
import docx
from docx.oxml.ns import qn
doc = docx.Document("/home/jd/metadata/curriculum-tracker.docx")
def iter_block_items(parent):
    for child in parent.element.body.iterchildren():
        if child.tag == qn('w:p'):
            yield docx.text.paragraph.Paragraph(child, parent)
        elif child.tag == qn('w:tbl'):
            yield docx.table.Table(child, parent)
for block in iter_block_items(doc):
    ... # paragraph text via block.text, or table rows via block.rows
EOF

```

This preserves paragraph/table order correctly (a naive regex split on the raw `word/document.xml` mis-handles nested table XML — don't do that).

38. CURRENT CHECKPOINT (authoritative — 2026-09-15)

Restarting at Week 1: Electricity, States, Bits, and Information (Stage 1).

Per explicit user instruction, prior "Chapter 1-6" work (§13, §34) is NOT credited against any tracker week — we begin the tracker from the top, in order, and will not skip ahead even where topics were already touched on informally.

Week 1 Learning Objectives (from the tracker, for reference):

- Understand voltage as a range rather than a perfect 0/1 value
- Explain why digital systems use stable state ranges
- Define bit, byte, word, encoding, and information
- Convert values between decimal, binary, and hexadecimal

Week 1 Hands-on Work (from the tracker):

- Convert 25 decimal values among decimal, binary, and hexadecimal
- Write a short explanation: "Why binary is physically practical"
- Create a one-page diagram: voltage → bit → pattern → meaning

Week 1 Mastery Checkpoint (from the tracker): *"Explain why the same bit pattern can represent a number, character, instruction, or pixel."*

Status: Week 1 in progress — teaching + hands-on work delivered in chat on 2026-09-15; user has not yet self-recorded dates/hours/confidence/status checkboxes in the .docx itself (see §36 step 6 — that's the user's own honest self-assessment to fill in, not something to auto-fill).

Next action for a new/continuing session: resume Week 1 evidence review (if not yet marked complete by the user in the .docx), then proceed to Week 2: Signed Numbers and Bitwise Reasoning.

39. DAILY LESSON WORKFLOW — 2026-09-16

The user explicitly chose this daily operating model:

1. Read the authoritative `.docx` tracker and identify the current week, objectives, hands-on work, and mastery checkpoint.
2. Create one focused daily lesson rather than compressing an entire tracker week into one response. Preserve the first-principles, WHY-before-HOW teaching style.
3. Save the lesson source under `daily/lessons/`.
4. Render a sequence-numbered `day-NNN-*` PDF under `daily/pdf/` and display/open it in the editor. Do not put dates or week numbers in daily artifact names; skipped calendar days must not create gaps in the learning sequence.

5. After the reading, provide hands-on reinforcement appropriate to the topic. This is optional per concept, not a mandatory HTML deliverable. Choose the most useful format:

- interactive HTML simulation or quiz,
 - coding/debugging exercise,
 - terminal experiment,
 - diagram/tracing exercise,
 - short conceptual quiz or prediction challenge.
6. Prefer **predict → run/observe → explain** over passive questions. Do not reveal solutions before the user attempts a challenge unless requested.

1. Use the challenge to gather evidence for the tracker's Explain / Draw / Observe / Build standard, but do not mark the user's confidence, hours, or mastery without their self-assessment.

2. Update this context file only at meaningful checkpoints; do not bloat it with the full text of every daily lesson.

Daily lesson writing tone

The user explicitly wants the notes to feel natural, engaging, and easy to read—not monotonic, bland, or like a mechanically generated textbook.

- Write like an experienced systems mentor talking through an idea at a whiteboard.
- Vary the rhythm: use questions, short explanations, concrete stories, diagrams, predictions, and occasional deeper technical passages.
- Build curiosity before presenting terminology. Introduce a problem first, then explain why the abstraction exists.
- Prefer plain language, but retain technical precision. Never make a concept inaccurate merely to make it sound simple.
- Connect abstract ideas to familiar machines, Linux behavior, and the learner's kernel/virtualization experience.
- Avoid repetitive templates, dense definition dumps, corporate prose, and long sequences of similarly shaped bullet points.
- Include moments where the learner pauses to predict an outcome or explain an idea in their own words.
- Enthusiasm should come from the ideas and discoveries, not from excessive exclamation marks or artificial praise.

On `master`, the approved weekly artifacts and exact resume point are indexed by `metadata/WEEKLY_PROGRESS.md`. Earlier daily approaches are preserved on `backup/narrow-daily` and `backup/accelerated-daily`.

40. WEEKLY MODULE WORKFLOW — 2026-09-23 (supersedes §39 on experiment)

The user now prefers weekly mastery modules rather than daily artifacts. On `master`:

- each module lives in a numbered topic folder such as `01-foundations-to-cpu/`;
- each topic contains `overview.md`, seven daily source sections under `sections/`, labs under `challenges/`, and `week-NN-material.pdf`;
- `metadata/WEEKLY_PROGRESS.md` stores the exact resume point;
- `tools/week` builds/opens the latest module, or `MC_WEEK=N tools/week` selects another.

The original accelerated Days 1–7 are daily sections inside Week 1 and are rendered into one weekly PDF without dropping content. “Week” describes a material package, not a deadline; a module may take as long as mastery requires. Earlier daily structures remain available only as backup branches. Metadata, tools, references, and archived prototypes are kept outside the learner-facing numbered topic folders.

41. APPROVED CPU/CHIP AND TOYOS TRACK — 2026-09-24

The learner is already familiar with much of the introductory material and wants the course to move beyond definitions into a physically grounded model of CPU/chip design.

After completing the three prepared review modules, create `04-cpu-and-chip-design/` and follow this dependency path:

MOSFET/CMOS

- restoring gates
- delay, capacitance, fan-out, power, and noise margins
- combinational/sequential timing
- SystemVerilog RTL and simulation
- synthesis and gate-level netlist
- ALU, register file, decoder, PC, and control
- simple RISC-V CPU
- pipeline/hazards/branch handling
- cache, TLB/MMU, and memory interface

Use ngspice for selected transistor-level experiments; Icarus or Verilator and GTKWave for RTL simulation; and Yosys for synthesis. The objective is a causal model of how a simple C computation becomes instruction bytes, control signals, gate transitions, and transistor-level charge/voltage changes—not professional analog IC specialization.

Then begin a guided x86-64 ToyOS build-along under QEMU:

1. freestanding kernel image, linker script, stack, serial output;
2. exception entry, IDT, and register dump;
3. physical page allocation;
4. page tables and virtual memory;
5. interrupts and timer;
6. task state and scheduling;
7. user mode and system calls;
8. processes/address spaces and ELF user loading;
9. descriptor table and minimal ramfs/VFS;
10. optional virtio/PCI/MMIO/DMA experiments.

Each milestone follows:

question → design → predict → implement → observe → break/debug
→ compare with xv6 → compare with Linux

The custom CPU track and ToyOS track meet at the ISA boundary. Do not start ToyOS by copying a finished template before assembly, ABI, linker/ELF, CPU state, privilege, and exception foundations are understood.

Scope correction

This is not an HDL or ASIC-specialization track. The learner wants only the hardware depth necessary for a systems programmer to understand CPU behavior. Prepared packages:

- `04-cpu-and-chip-design/` — transistor/gate/timing foundations, one small RTL exercise, and one synthesis observation;
- `05-riscv-cpu-microarchitecture/` — simple RV32I datapath/control, runnable core model, pipeline/hazards, cache/TLB/MMU, and end-to-end task trace.

Return to OS/kernel construction after these modules. Treat analog circuit design, fabrication, advanced verification, and physical-design expertise as out of scope unless a later systems question genuinely requires them.

42. FUNDAMENTALS-FIRST GATES AND CAREER PIVOT — 2026-09-30

`metadata/COURSE_ROADMAP.md` is the canonical strategic plan. The original 66-week tracker remains authoritative for detailed topic objectives, but active progression is controlled by confidence gates:

- **Gate A:** CPU, C, assembly, ABI, memory hierarchy, toolchain, ELF, and debugging clarity.
- **Gate B:** transistor/gate/timing-to-CPU boundary, minimal RTL/synthesis, microarchitecture, pipeline, cache/TLB/MMU, and end-to-end task trace.
- **Gate C:** syscall/exception/interrupt/scheduler/page-fault paths plus simplified OS mechanisms and ToyOS/xv6/Linux comparison.

Required continuous lanes:

- systems-oriented C coding and toolchain inspection;
- user-space and kernel-level debugging;
- core-kernel execution-path atlas;
- deep CPU-scheduler mental model and tracing;
- interview explanation/coding/debugging practice;
- cross-layer kernel systems topics such as ordering, NUMA, PCIe/DMA/IOMMU, storage, networking, virtualization, isolation, and observability.

Career specialization is adaptive but deferred until foundations. KVM/x86/AMD virtualization remains the default differentiator inside a broad kernel/platform profile. Networking/eBPF, embedded/BSP, PCIe/accelerator, and storage paths are tested later with bounded proof-of-fit sprints and real job market evidence.

Upstream contribution is a required final outcome, not an immediate excuse to skip foundations. During foundation work: read lore, build/test upstream kernels, and review patches. After Gate C: select/reproduce a real issue, implement/test a focused fix, submit it publicly, and respond to maintainer review.